

P0737r0 : Execution Context of Execution Agents

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0737r0
Date: 2017-10-11
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards, Sandia National Laboratories
Author: Daniel Sunderland, Sandia National Laboratories
Author: Michael Wong, Codeplay
Author: Thomas Rodgers
Author: Gordon Brown, Codeplay
Audience: SG1 Concurrency and Parallelism
URL: https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0737_ExecContext.rst

Revision History

D0737r0 : For 2017-11-Albuquerque pre-meeting mailing

- State: Design exploration
- Requested straw polls:
 - Proposed definition of *ExecutionContext concept*
 - Non-copyable and non-moveable
 - Execution resource
 - Wait functions
 - Executor generator
 - Query property
 - Destructor behavior
 - Proposed thread execution resource
 - Proposed `this_thread` query of thread execution resource
 - Proposed standard async execution context and executor
 - Interest in each of the suggested [potential additions](#) for initial insertion into Concurrency or Executors TS

The Problem

Current Executor proposal does not define a specific concrete context. An Execution Context is meant to mediate between Execution agents, an executor, and execution resources. Specifically, an execution context is responsible for managing the execution of a number of (light-weight) execution agents on an execution resource. An execution context also provides executors for submitting work to itself.

The Executor proposal currently does not specifically define a concrete execution context, other than providing a static thread pool as a basic example. As the executor proposal is a joint proposal between several industry representatives - parallel and vectorized algorithms, multi-threaded execution, heterogeneous and distributed execution, and network execution. It was not deemed necessary to provide a specific concrete execution context. For some areas, a concrete execution context is necessary because it could be used to manage a stream or queue of command kernels, such as in heterogeneous or distributed computing. In other domains, such as parallel and vectorized algorithms, it may remain an abstract concept.

This paper focuses on those domains where a concrete Execution Context is extremely important while also proposing a mechanism for defining the system affinity.

Another area of interest that can be supported by Context is the querying of the memory affinity status of the system. This is important as a concept to enable future support for affinity (see the Affinity paper). This is an area that requires solution and this paper outlines one such direction.

Proposal

Add this minimal ExecutionContext specification to the Executors TS.

Concept / Definition

A concurrency and parallelism **execution context** manages a set of execution agents on a set of **execution resources** of a given **execution architecture**. These execution agents execute work, implemented by a callable, that is submitted to the execution context by an **executor**. One or more types of executors may submit work to the same execution context. Work submitted to an execution context is **incomplete** until (1) it is invoked and exits execution by return or exception or (2) its submission for execution is cancelled.

Note: The *execution context* terminology used here and in the Networking TS (N4656) deviate from the traditional *context of execution* usage that refers to the state of a single executing callable; *e.g.*, program counter, registers, stack frame.

Contrast to Networking TS (N4656) Execution Context

The Networking TS (N4656 / 2017-03-17) defines a *networking execution context* as

"Class `execution_context` implements an extensible, type-safe, polymorphic set of services, indexed by service type."

The networking TS requirements for `execution_context` specify a single `execution_context::executor_type`. This executor type has numerous work submission member functions each with particular semantics.

Differences between our proposed parallelism and concurrency execution context and the networking execution context include the following.

1. Networking context is Limited to executing work, as opposed to providing unspecified services.
2. Networking context is not a concrete base class from which other forms of execution contexts are derived; *e.g.*, `system_context`, `io_context`.
3. Our context is an extensible one-to-many relationship between an execution context type and executor types that may submit work, as opposed to a particular executor type with specific work submission functions.
4. Our proposed `async_execution_context` could be viewed as having similar intent as the networking `system_context`. The significant difference is interchangeability with `std::async` usage and extensibility to other executors versus the prescribed networking `system_executor`.

Partitioning and Affinity State of the Art

Hardware Locality (hwloc) Software Package

The [hardware locality \(hwloc\) software package](#) provides a portable interface for querying and managing hardware resources, including *processing units* on which threads execute. The proposed thread execution resource is motivated by a small fraction of hwloc capabilities.

SYCL for OpenCL and HSA

[SYCL](#) (based on [OpenCL](#)) provides a C++ programming model for targeting a wide range of heterogeneous systems including multi-core CPUs, NUMA systems, embedded SoCs and discrete accelerators. [HSA \(Heterogeneous System Architecture\)](#) is a similar standard that provides a lower level, and lower overhead API and set of architecture requirements.

Both of these standards represent the topology of a system with a hierarchy of ids that remain constant throughout the execution of a program. Both also allow users to partition the system topology to do fine-grained work execution. The extent of the partitioning depends on the platform.

OpenMP

In contrast, OpenMP requires an external environment variable set by the user. They use the idea of an abstract Place as defined by the user over all threads, cores, and sockets. In this way, it enable the user to specify the granularity of the topology, and then further enable defining the desired affinity for being on the same Place as master thread, or scatter out in a round robin fashion, as well as compacting it around the Master. It can also define for each nested parallelism level, because the work may change or become irregular during runtime.

This design, while flexible is not possible for C++ Affinity. C++ cannot use the environment variable for input configuration. The advantage of the OpenMP design is its use of abstract places which makes it flexible for some future core configuration but it means the programmer has to decide whether to describe the places in terms of threads, cores, or sockets which still requires some actual hardware knowledge. Still, the fundamental of its implementation has shown it is doable on most architectures. In that respect, C++ implementation would only need to define the interface, but the underlying mechanism should be similar. Since its addition in OpenMP 3, this feature has a great deal of experience from HPC and demonstrates implementability.

Minimal Concept Specification

The proposed *parallelism and concurrency execution context* has minimal scope, with the intent to grow this scope as consensus is achieved on [potential additions](#).

```
class ExecutionContext /* exposition only */ {
public:

    template <typename ExecutionContextProperty>
        /* exposition only */ detail::query_t< ExecutionContext , ExecutionContextProperty >
        query(ExecutionContextProperty p) const ;

    ~ExecutionContext();

    // Not copyable or moveable
    ExecutionContext( ExecutionContext const & ) = delete ;
    ExecutionContext( ExecutionContext && ) = delete ;
    ExecutionContext & operator = ( ExecutionContext const & ) = delete ;
    ExecutionContext & operator = ( ExecutionContext && ) = delete ;

    // Execution resource
    using execution_resource_t = /* implementation defined */ ;

    execution_resource_t const & execution_resource() const noexcept ;
```

```

// Executor generator
template< class ... ExecutorProperties >
    /* exposition only */ detail::executor_t< ExecutionContext , ExecutorProperties... >
    executor( ExecutorProperties... );

// Waiting functions:
void wait();
template< class Clock , class Duration >
bool wait_until( chrono::time_point<Clock,Duration> const & );
template< class Rep , class Period >
bool wait_for( chrono::duration<Rep,Period> const & );
};

bool operator == ( ExecutionContext const & , ExecutionContext const & );
bool operator != ( ExecutionContext const & , ExecutionContext const & );

// Execution context properties:

constexpr struct reject_on_destruction_t {} reject_on_destruction;
constexpr struct abandon_on_destruction_t {} abandon_on_destruction;
constexpr struct abort_on_destruction_t {} abort_on_destruction;
constexpr struct wait_on_destruction_t {} wait_on_destruction;

```

Let EC be an *ExecutionContext* type.

```

template <typename ExecutionContextProperty>
    /* exposition only */ detail::query_t< EC , ExecutionContextProperty >
    EC::query(ExecutionContextProperty p) const noexcept;

```

Requires: *ExecutionContextProperty* is an execution context property.

Returns: The current value of the execution context property specified by *p* when the execution context supports the input property. Otherwise void. [Note: The *detail::query_t* is for exposition only denoting the expectation that an implementation will use an implementation-defined metafunction to determine the return type. --end note]

Remark: When `is_same_v<void,decltype(ec.query(p))` then the execution context property is unknown to the execution context.

```

EC::execution_resource_t const & EC::execution_resource() const noexcept ;

```

Returns: A descriptor of the execution resource(s) utilized by this execution context to execute work. Execution architecture is identified by the *execution_resource_t* type.

```

template< class ... ExecutorProperties >
    /* exposition only */ detail::executor_t< EC , ExecutorProperties... >
    EC::executor( ExecutorProperties ... p );

```

Returns: An executor with *this execution context and execution properties *p* when the execution context supports these properties. Otherwise void. [Note: The *detail::executor_t* is for exposition only denoting the expectation that an implementation will use an implementation-defined metafunction to determine the type of the returned executor. --end note]

```

static_assert( ! is_same_v< void , decltype( ec.executor( p... ) )
    , "Execution context cannot generate executor for given executor properties." );

```

Remark: A particular execution property may have semantic and interface implications, such as whether application of the executor returns a future or not (sometimes referred to as a two-way or one-way property). A particular execution property may only be a performance hint.

```

void EC::wait();

```

Requires: Cannot be called from non-blocking work submitted to this execution context. [Note: Work waiting upon itself guarantees deadlock. --end note]

Effects: Waits until the number of incomplete, non-blocking callables submitted to the execution context is observed to be zero. [Note: The execution agent from which the wait function is called should *boost block* execution agents in the execution context. --end note]

```

template< class Clock , class Duration >
bool EC::wait_until( chrono::time_point<Clock,Duration> const & dt );
template< class Rep , class Period >
bool EC::wait_for( chrono::duration<Rep,Period> const & dt );

```

Requires: Cannot be called from non-blocking work submitted to this execution context. [Note: Work waiting upon itself can never return true. --end note]

Returns: true if the number of incomplete callables is observed zero at any point during the call to wait.

Effects: Waits at least *dt* for the number of incomplete, non-blocking callables submitted to the execution context is observed to be zero. [Note: The execution agent from which the wait function is called should *boost block* execution agents in the execution context, but may only poll to honor the time out. --end note]

```

EC::~~EC();

```

Effects: Behavior during destruction is denoted by the values of the queried execution context properties.

- If `EC::query(reject_on_destruction)` returns true then submission of any new work to `*this` is rejected.
- If `EC::query(abandon_on_destruction)` returns true then work that has been submitted to `*this` but has not yet started executing is abandoned and submission of any new work to `*this` is rejected.
- If `EC::query(wait_on_destruction)` returns true then `*this` destructor blocks until work that is executing, not rejected, or not abandoned has completed.
- If `EC::query(abort_on_destruction)` returns true then `*this` attempts to abort work that is executing, abandons work that has not yet started executing, rejects submission of any new work, and does not block. It may not be feasible to abort executing work; however, such work will cease to have a defined execution context.

Remark: For example, if `EC::query(reject_on_destruction)`, `!EC::query(abandon_on_destruction)`, and `EC::query(wait_on_destruction)` then submission of new work is rejected and the destructor blocks until all executing and previously submitted work completes.

```
bool operator == ( EC const & lhs , EC const & rhs );
bool operator != ( EC const & lhs , EC const & rhs );
```

Effects: Return whether `lhs` and `rhs` refer (or not) to the same execution context: submitting work to `lhs` has identical effect as submitting work to `rhs`.

Execution Resource (see also P0761, Executors Design Document)

An *execution resource* is an descriptor for an implementation defined hardware and/or software facility capable of executing a callable function object. Different resources may offer a broad array of functionality and semantics and exhibit different performance characteristics of interest to the performance-conscious programmer. For example, an implementation might expose different processor cores, with potentially non-uniform access to memory, as separate resources to enable programmers to reason about locality.

An execution resource can range from SIMD vector units accessible in a single thread to an entire runtime managing a large collection of threads.

Thread Execution Resource

A *thread of execution* executes on a *processing unit* (PU) within an *execution resource*. *Threads of execution* can make *concurrent forward progress* only if they execute on different processing units. Conversely, a single processing unit cannot cause two or more *threads of execution* to make concurrent forward progress. A *thread execution resource* is associated with a specific set of processing units within the system hardware.

[Note: A *CPU hyperthread* is a common example of a processing unit. In a Linux runtime a *thread execution resource* is defined by a `cpu_set_t` object and is queried through the `sched_getaffinity` function. --end note]

A *processing unit* or *thread execution resource* may be what was intended by the undefined term "thread contexts" in 33.3.2.6, "thread static members."

A *thread execution resource* may have *locality partitions* for its associated set of processing units. For example, hyperthreads sharing the same CPU core are more local to one another than to a hyperthreads on different core.

```
struct thread_execution_resource_t {
    thread_execution_resource_t() = delete ;
    thread_execution_resource_t( thread_execution_resource_t && ) = delete ;
    thread_execution_resource_t( thread_execution_resource_t const & ) = delete ;
    thread_execution_resource_t & operator=( thread_execution_resource_t && ) = delete ;
    thread_execution_resource_t & operator=( thread_execution_resource_t const & ) = delete ;

    size_t concurrency() const noexcept ;

    size_t partition_size() const noexcept ;

    const thread_execution_resource_t & partition( size_t i ) const noexcept ;

    const thread_execution_resource_t & member_of() const noexcept ;
};

extern thread_execution_resource_t program_thread_execution_resource ;
```

```
size_t concurrency();
```

Returns: This execution resource's potential for concurrent forward progress; *i.e.*, the number of processing units associated with this execution resource.

Remark: Has similar intent as 33.2.2.6 `std::thread::hardware_concurrency()`; which returns "The number of hardware thread contexts."

```
size_t partition_size() const noexcept ;
```

Returns: Number of locality partitionings of the execution resource.

```
const thread_execution_resource_t & partition(size_t i) const noexcept ;
```

Requires: `i < partition_size()`.

Returns: A locality partition of the execution resource. Locality partitions are associated disjoint subsets of the thread execution resource's processing units.

```
void verify_concurrency( thread_execution_resource_t const & E )
{
    size_t sum = 0 ;
    for ( size_t i = 0 ; i < E.partition_size() ; ++i )
        sum += E.partition(i).concurrency();
    assert( E.partition_size() == 0 || E.concurrency() == sum );
}
```

Remark: Processing units residing in the same locality partition are *more local* with respect to the memory system than processing units in disjoint partitions. For example, non-uniform memory access (NUMA) partitions.

```
const thread_execution_resource_t & member_of() const noexcept ;
```

Returns: If thread execution resource `M` is a member of a thread execution resource `E` partitioning then returns `E`, `M == E.partition(i)` for some `i` then `E == M.member_of()`. Otherwise returns `M`.

```
extern thread_execution_resource_t program_thread_execution_resource ;
```

Thread execution resource in which the program is *permitted* to execute threads. When a program executes it is common for the system runtime to restrict that program to execute on a subset of all possible processing units of the system hardware.

[Note: For example, the linux taskset command can restrict a program to a specified set of processing units. The program can use `sched_getaffinity(0, ...)` to query that restriction. The proposed `program_thread_execution_resource` is intended to provide the same information. --end note]

Requires: `program_thread_execution_resource.member_of() == program_thread_execution_resource` and all `member_of()` recursions terminate with `program_thread_execution_resource`.

Remark: A high-quality implementation will provide a hierarchical locality partitioning that terminates when members have `concurrency() == 1`.

This Thread Execution Resource

Add to 33.3.3 Namespace `this_thread`

```
namespace std::this_thread {
    const thread_execution_resource_t & get_resource();
}
```

```
const thread_execution_resource_t & this_thread::get_resource()
```

Returns: An execution resource on which this thread was executing during the call to `get_resource`.

Remark: A thread may migrate between thread execution resources. As such the `get_resource` returns one of those resources on which the thread was executing during the call to `get_resource`. There is no guarantee that this thread is executing on the returned thread execution resource before or after the call to `get_resource`. A high-quality implementation will return an execution resource with `concurrency() == 1`.

Motivation for Standard Async Execution Context and Executor

Require that the 33.6.9 Function template `async` have an equivalent execution context and executor based mechanism for launching asynchronous work. This exposes the currently hidden execution context and executor(s) which the underlying runtime has implemented to enable `std::async`.

```
// Equivalent without- and with-executor async statements without launch policy
auto f = std::async( []{ std::cout << "anonymous way\n" } );
auto f = std::async( std::async_execution_context.executor() , []{ std::cout << "executor way\n" } );

// Equivalent without- and with-executor async statements with launch policy
auto f = std::async( std::launch::deferred , []{ std::cout << "anonymous way\n" } );
auto f = std::async( std::async_execution_context.executor( std::launch::deferred ) , []{ std::cout << "executor way\n" } );
```

Wording for Standard Async Execution Context and Executor

```
namespace std {
    struct async_execution_context_t {
        // conforming to ExecutionContext concept
```

```

// Execution resource
using execution_resource_t = thread_execution_resource_t ;

template< class ... ExecutorProperties >
/* exposition only */ detail::executor_t< async_execution_context_t , ExecutorProperties... >
executor( ExecutorProperties ... p );
};

class async_executor_t ; // implementation defined

extern async_execution_context_t async_execution_context ;

template< class Function , class ... Args >
future<std::result_of<std::decay_t<Function>(std::decay_t<Args>...)>>
async( async_executor_t exec , Function && f , Args && ... args );
}

```

```
extern async_execution_context_t async_execution_context;
```

Global execution context object enabling the equivalent invocation of callables through the with-executor `std::async` and without-executor `std::async`. Guaranteed to be initialized during or before the first use. [Note: It is likely that `async_execution_context == program_thread_execution_context`. --end note]

```

template< class ... ExecutorProperties >
/* exposition only */ detail::executor_t< async_execution_context_t , ExecutorProperties... >
async_execution_context_t::executor( ExecutorProperties ... p );

```

Returns: An *executor* with *this execution context* and execution properties *p*. If *p* is empty, is `std::launch::async`, or is `std::launch::deferred` the *executor* type is `async_executor_t`.

```

template< class Function , class ... Args >
future<std::result_of<std::decay_t<Function>(std::decay_t<Args>...)>>
async( async_executor_t exec , Function && f , Args && ... args );

```

Effects: If `exec` has a `std::launch` *policy* then equivalent to invoking `std::async( policy , f , args... )`; otherwise equivalent to invoking `std::async( f , args... )`; Equivalency is symmetric with respect to the non-executor `std::async` functions.

Appendices

Potential Additions: Request straw poll for each

Straw polls requested for each of the following potential additions.

- Strongly-favor = I must have this in next revision of this paper.
 - Weakly-favor = I'd like to see this in a future paper, or perhaps the next revision.
 - Neutral = *whatever*
 - Weakly-against = I don't want to see this in the next revision of this paper, but I am willing to look at it in a future paper.
 - Strongly-against = I never want to see this in any paper.
1. Add to *thread_execution_resource_t* a hardware architecture trait; e.g., the **hwloc** trait for *socket*, *numa*, and *core*.
 2. A mechanism to bind the execution of a `std::thread` to a specified `thread_execution_resource`. Note that by definition all `std::thread` are bound to `program_thread_execution_resource`.
 3. A mechanism to accumulate and query exceptions thrown by callables that were submitted by a one-way executor.
 4. A mechanism to provide a callable that is invoked to consume exceptions thrown by callables that were submitted by a one-way executor.
 5. A mechanism for preventing further submissions. Related to "closed" in Concurrent Queue paper P0260.
 6. A mechanism for cancelling submitted callables that have not been invoked. Similar intent as Networking TS `system_executor::stop()`. Note that an execution resource and associated execution context may not be able to cancel submitted work; e.g., submitted to a hardware queue or to a remote dispatch-and-forget queue.
 7. A mechanism for aborting callables that are executing. *Included for completeness, the authors are strongly-against.*
 8. A preferred-locality (affinity) memory space allocator
 9. Proposal to revise Networking TS execution context to align with parallelism and concurrency execution context.
 10. Current *thread_execution_resource_t* assumes static set of processing units with static hierarchical partitioning topology. A process' set of processing units and associated topology could be dynamic such that an executing process could adapt to changes; e.g., cores could dynamically go off-line and previously off-line cores could come back on-line. A dynamic set of processing units and dynamic hierarchical partitioning topology would require a complete redesign to address race conditions between querying and changing the execution resource. *Authors need to see a performant runtime that handles such dynamicity before considering such a change.*